

Can This Agent Even Do That?

A Decidable Goal-Achievability Type Discipline for LLM-Synthesized Agent Skills

Anonymous Author(s)

Affiliation · Draft (technical core) · a mechanized artifact accompanies this paper

Abstract

Autonomous agents are increasingly configured by *skills*: natural-language specifications that an LLM drafts from a user’s stated intent and then executes, often across several cooperating agents. Such drafts fail expensively at run time on plans that were impossible from the start — a step that invokes a tool the agent does not have, a goal conjunct nothing can establish, a budget no run can meet, or a multi-agent handoff that deadlocks. We present a *skill compiler* that decides, before execution and without an LLM judge, whether a skill’s *goal is achievable at all*. The compiler fuses two mature disciplines not previously combined for this purpose: capability-guarded reachability from automated planning, and projectability and deadlock-freedom from multiparty session types, decided over an intent-synthesized, goal-marked global type. The LLM that compacts prose into a formal pack of preconditions, effects, and goals is treated as *untrusted*; the checker is a mechanically verified core. We prove in Coq that the checker is *sound for refutation* — a verdict of *impossible* is never wrong — while being deliberately *incomplete* for achievability, with the residue confined to two precisely characterized edges (payload faithfulness and intent fidelity) handed respectively to a runtime monitor and a human checkpoint. We give the syntax, an operational semantics, the typing and achievability judgments with inference rules, and the metatheory: refutation soundness, tolerance soundness, capability monotonicity, decidability within a static-topology fragment, and undecidability once agents may spawn participants at run time. An implementation and a corpus spanning the canonical agent failure modes confirm the theory: zero false impossibilities, and every false achievability an instance of the two deferred edges. This extended version adds four implementation-driven strengthenings with proofs — a protocol-independent *establisher-closure* refutation that survives the undecidable autonomous fragment, *observed choice* for conversation-embedded coordination, an *adversarial* (must-)achievability mode whose refutations inherit soundness compositionally, and counterexample-guided compaction repair — and an evaluation on a corpus of real, deployed skills: every deployed skill checks achievable in its home capability context, every seeded semantic fault is refuted with the missing tool or dead goal conjunct named.

1 Introduction

A modern agent is rarely a single model call. It is a *skill*: a specification — increasingly authored in natural language and drafted by an LLM from a user’s stated goal — that names tools, describes a plan, and coordinates one or more agents toward an outcome. The appeal is that a non-expert can say *what* they want and have the system assemble the *how*. The hazard is that the assembled how is frequently *impossible*, and nothing notices until execution has already spent tokens, taken actions, and, in the multi-agent case, deadlocked halfway.

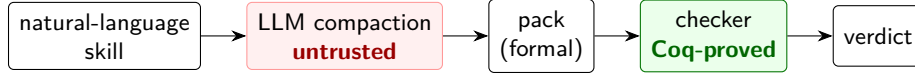
The failures are structural and recurring. A plan *books a flight and emails a confirmation* although no email tool is in scope (a hallucinated capability). A plan pursues *a flight under \$500* when the only booking tool returns premium fares (an unsatisfiable refinement). A planner waits for a worker’s result while the worker waits for the planner’s answer, each listening for a message the other never sends (a coordination deadlock). In every case the goal was unreachable *a priori*, and a cheap static check could have said so.

This paper asks a deliberately narrow question and answers it with a guarantee: **given an LLM-drafted skill, can its goal be achieved at all?** We are not (here) verifying that every

step is safe, nor that the agent behaves correctly on every input. We decide *achievability*, tolerantly — small details, detours, and payload variation must not raise a false alarm — and *soundly for refutation*: when the compiler says a goal is impossible, it is right.

1.1 The idea, and the trust boundary

The architecture has two halves separated by a trust boundary. An LLM performs *compaction*: it reads the prose skill and distills it into a formal *pack* — capabilities annotated with preconditions and effects, a goal-marked global protocol, and an initial state. This step is *untrusted*; the LLM may misread the prose. A deterministic *checker* then decides achievability over the pack, and the checker is mechanically proved sound. The division is what makes the system honest: a faulty compaction can only ever cause a false *achievable* (caught downstream), never a false *impossible* about the pack it received.



Crucially, the semantic gap between *what the user meant* and *what was formalized* is not eliminated — it is *relocated and concentrated* into one inspectable checkpoint at compaction time, rather than scattered, unobserved, across every run. That relocation is the central design move.

1.2 Contributions

1. **A goal-achievability type discipline** (§3–§5) that fuses capability-guarded reachability (planning) with projectability and deadlock-freedom (multiparty session types), decided over an intent-synthesized, goal-marked global type. To our knowledge this fusion, aimed at LLM-drafted agent skills, is new.
2. **A mechanized soundness core** (§6) in Coq: refutation soundness (a verdict of impossible is never wrong), tolerance soundness (coarsening the abstraction preserves refutation), and capability monotonicity (more tools never make a goal impossible) — all axiom-free.
3. **A decidability boundary result** (§6.4): achievability is decidable in a static-topology fragment and undecidable once agents may spawn participants at run time. Autonomy is exactly what crosses the line.
4. **An implementation and evaluation** (§8–§9): a checker deciding all four premises of the judgment (capability soundness, projection with merge, Gay–Hole conformance, may-reachability), an LLM-compaction front-end with a schema gate, and a corpus spanning the canonical failure modes. Results match the theory: zero false impossibilities; every false achievability is one of the two deferred residues.
5. **Implementation-driven strengthenings, with proofs** (§7): the establisher-closure certificate (protocol-independent refutation, valid even under dynamic spawning), observed choice (conversation-embedded selection is an implicit broadcast; unobserved choice is a ≥ 3 -party asynchronous phenomenon), adversarial must-achievability (refutation soundness inherited compositionally from Theorem 1), and counterexample-guided compaction repair at the trust boundary.
6. **Validation on real skills** (§9): on Anthropic’s public skills corpus, the compiler judges every deployed skill achievable in its home capability context (no false alarms), refutes the same skills under a capability-poorer runtime with the exact missing tool and source line, and catches every seeded semantic fault (dropped capability, stripped goal establisher) naming the wound.

2 Overview by Example

Consider a one-agent skill whose goal is *a flight is booked and a confirmation is sent*. The author writes prose; the LLM compacts it. Two compactations differ only in whether an email tool is in scope.

$$\varphi_{goal} = \text{booked} \wedge \text{confirmation_sent}$$

variant A (no email tool)	variant B
search : <i>Add</i> {searched}	... A, plus:
filter : <i>pre</i> searched; <i>Add</i> {filtered}	email : <i>pre</i> booked;
book : <i>pre</i> filtered; <i>Add</i> {booked}	<i>Add</i> {confirmation_sent}

Under STRIPS *frame semantics* — a predicate is false unless an effect makes it true — variant A has no action establishing `confirmation_sent`, so no reachable state satisfies the goal: the checker returns *impossible* and points at the missing establisher. Variant B reaches the goal along `search · filter · book · email`, returned as a witness. The same example is mechanized in §6 (the concrete instance `FlightInstance`): variant A is *provably* unachievable, yet a booking is still reachable — the refutation is about the missing tool, not a dead protocol.

Tolerance is essential, and is where a naive reachability check would misfire. Extra status messages, a clarifying detour, or payload variation must not refute the goal. We obtain this from three orthogonal relaxations made precise in §4–§5: *existential may-reachability* (a single good path suffices), *session subtyping* (interface slack in the safe direction), and *goal-relevant abstraction* (payload detail collapsed before checking).

3 Syntax

We fix disjoint sets of predicates $p \in \text{Pred}$ (boolean), numeric variables $x \in \text{Var}$, roles $p, q \in \mathcal{R}$, capability names $a \in \text{Cap}$, and message labels $\ell \in \text{Lab}$.

3.1 State logic

$$\begin{aligned} e &::= x \mid n \mid e + e \mid e - e \mid c \cdot e & (c, n \in \mathbb{Z}) \\ \varphi &::= p \mid \top \mid \perp \mid \neg \varphi \mid \varphi \wedge \varphi \mid \varphi \vee \varphi \mid e \bowtie e & \bowtie \in \{<, \leq, =, \geq, >, \neq\} \end{aligned}$$

The state logic $\mathcal{L}$ is quantifier-free linear integer arithmetic with uninterpreted boolean predicates — a decidable fragment for which entailment $\models$ and satisfiability are discharged by an SMT solver.

3.2 Worlds and capabilities

A *world* $W = \langle B, N \rangle$ evaluates predicates $B : \text{Pred} \rightarrow \mathbb{B}$ and numeric variables $N : \text{Var} \rightarrow \mathbb{Z}$. A *capability context* Γ maps each available tool a to a guarded effect:

$$\Gamma(a) = \langle pre_a, eff_a \rangle, \quad pre_a \in \mathcal{L}, \quad eff_a = \langle Add_a \subseteq \text{Pred}, Del_a \subseteq \text{Pred}, Asg_a : \text{Var} \rightarrow e, Nd_a : \text{Var} \rightarrow \mathcal{L} \rangle$$

$$a \notin \text{dom}(\Gamma) \iff \text{the agent does not possess tool } a \quad (\text{no hallucinated tools}).$$

Add/Del are the STRIPS add- and delete-lists; *Asg* gives deterministic numeric updates $x := e$; *Nd* gives *nondeterministic* updates $x := *$ constrained by a formula over the new value (used to model post-conditions such as “*books a fare under 500*”).

3.3 Global types, local types, skills

A *goal-marked global type* G describes the whole choreography; *local types* T are its per-role projections; a *skill* S_p is the behaviour the author declares for role p .

$G ::=$	$p \rightarrow q : \ell \langle \hat{\alpha} \rangle . G$	message (p sends ℓ to q , payload abstracted)
	$ \quad p \triangleright \{\ell_i : G_i\}_{i \in I}$	choice (p selects a branch)
	$ \quad a @ p . G$	action (role p invokes capability a)
	$ \quad \checkmark \varphi . G$	goal marker (the protocol claims φ holds)
	$ \quad \mu X . G \mid X \mid \mathbf{end}$	
$T ::=$	$q ! \ell \langle \hat{\alpha} \rangle . T \mid q ? \ell \langle \hat{\alpha} \rangle . T \mid \oplus \{\ell_i : T_i\} \mid \& \{\ell_i : T_i\}$	
	$ \quad a . T \mid \checkmark \varphi . T \mid \mu X . T \mid X \mid \mathbf{end}$	
$S_p ::=$	(same grammar as T)	the declared behaviour of role p

The abstraction α maps concrete messages to goal-relevant tokens $\hat{\alpha}$; it is the tolerance dial of §4.4. Only $a @ p$ steps change the world; messages carry coordination information and are abstracted for achievability.

4 Operational Semantics

4.1 World transitions

A capability fires when its guard holds, producing a successor world. Because Nd is nondeterministic, $\langle W \rangle a \langle W' \rangle$ may relate one world to many.

$$\frac{\text{WORLD-ACT} \quad W \models pre_a \quad W' \in \text{apply}(eff_a, W)}{\langle W \rangle a \langle W' \rangle}$$

$$\text{apply}(eff_a, \langle B, N \rangle) = \langle B', N' \rangle, \quad B' = (B \cup Add_a) \setminus Del_a$$

$$N'(x) = \begin{cases} \llbracket Asg_a(x) \rrbracket_N & \text{if } x \in \text{dom}(Asg_a) \\ v \text{ with } \langle B, N[x \mapsto v] \rangle \models Nd_a(x) & \text{if } x \in \text{dom}(Nd_a) \\ N(x) & \text{otherwise (frame)} \end{cases}$$

4.2 Configurations and may-reachability

A *configuration* is a pair (G, W) . The transition $\rightsquigarrow$ advances control, firing world transitions at actions and branching at choices; $\rightsquigarrow^*$ is its reflexive-transitive closure.

$$\begin{array}{c} \text{R-ACT} \\ \frac{\langle W \rangle a \langle W' \rangle}{(a @ p . G, W) \rightsquigarrow (G, W')} \end{array} \quad \begin{array}{c} \text{R-MSG} \\ \frac{}{(p \rightarrow q : \ell \langle \hat{\alpha} \rangle . G, W) \rightsquigarrow (G, W)} \end{array} \quad \begin{array}{c} \text{R-SEL} \\ \frac{k \in I}{(p \triangleright \{\ell_i : G_i\}_{i \in I}, W) \rightsquigarrow (G_k, W)} \end{array}$$

$$\begin{array}{c} \text{R-REC} \\ \frac{}{(\mu X . G, W) \rightsquigarrow (G[\mu X . G / X], W)} \end{array}$$

A configuration is *goal-satisfying* when control is at a goal marker or terminus and the world entails the goal:

$$\text{goal}(\checkmark \varphi . G, W) \iff W \models \varphi, \quad \text{goal}(\mathbf{end}, W) \iff W \models \varphi_{\text{goal}}.$$

$$\Gamma; G \models \Diamond \varphi_{\text{goal}} \iff \exists W_0 \models \text{init}. \exists (G', W'). (G, W_0) \rightsquigarrow^* (G', W') \wedge \text{goal}(G', W').$$

The diamond is *existential*: a single witnessing run suffices. This is the formal source of detour-tolerance.

4.3 Projection and realizability

Projection $G \upharpoonright \mathbf{p}$ extracts role $\mathbf{p}$'s local type, using the merge $\sqcap$ on continuations of a choice in which $\mathbf{p}$ is not the selector. It is *partial*: undefined exactly when a non-selecting role cannot tell which branch was taken (the *unobserved-choice* condition).

$$\begin{array}{c}
\text{PROJ-SEL} \\
\frac{G_i \upharpoonright \mathbf{p} = T_i \ (i \in I)}{(\mathbf{p} \triangleright \{\ell_i : G_i\}) \upharpoonright \mathbf{p} = \oplus \{\ell_i : T_i\}}
\end{array}
\qquad
\begin{array}{c}
\text{PROJ-BRN} \\
\frac{G_i \upharpoonright \mathbf{q} = T_i \ (i \in I) \quad \mathbf{p} \text{ sends a distinguishing } \ell_i \text{ to } \mathbf{q}}{(\mathbf{p} \triangleright \{\ell_i : G_i\}) \upharpoonright \mathbf{q} = \& \{\ell_i : T_i\}}
\end{array}$$

$$\begin{array}{c}
\text{PROJ-MRG} \\
\frac{G_i \upharpoonright \mathbf{q} = T_i \ (i \in I) \quad T = \sqcap_i T_i \text{ defined}}{(\mathbf{p} \triangleright \{\ell_i : G_i\}) \upharpoonright \mathbf{q} = T}
\end{array}$$

G is *realizable*, $G \Downarrow \{T_{\mathbf{p}}\}$, when $G \upharpoonright \mathbf{p}$ is defined for every role. Non-realizability is the static signature of a deadlocking handoff.

Observed choice. Real skills are frequently *one conversation*: an assistant and its user in a chat, or an agent and a business on a phone call. On such a medium the outcome of a choice is announced by the medium itself — every participant hears which way the call went. We therefore allow a choice to be annotated *observed*, $\mathbf{p} \triangleright^{\text{obs}} \{\ell_i : G_i\}$, projecting for a non-selector as an external choice on the selection labels directly:

$$\begin{array}{c}
\text{PROJ-OBS} \\
\frac{G_i \upharpoonright \mathbf{q} = T_i \ (i \in I) \quad \mathbf{q} \neq \mathbf{p}}{(\mathbf{p} \triangleright^{\text{obs}} \{\ell_i : G_i\}) \upharpoonright \mathbf{q} = \& \{\ell_i : T_i\}}
\end{array}$$

(when all T_i coincide the merge applies as usual and the annotation is inert). Section 7.1 proves this sound by desugaring into explicit broadcasts and observes that the unobserved-choice condition is intrinsically a ≥ 3 -party *asynchronous* phenomenon.

4.4 Subtyping and the tolerance dial

We use Gay–Hole branching subtyping $\leq$: a skill may offer *more* external (receive) choices and *fewer* internal (send) choices than its contract and remain safe.

$$\begin{array}{c}
\text{SUB-EXT} \\
\frac{I \supseteq J \quad T_i \leq T'_i \ (i \in J)}{\& \{\ell_i : T_i\}_{i \in I} \leq \& \{\ell_j : T'_j\}_{j \in J}}
\end{array}
\qquad
\begin{array}{c}
\text{SUB-INT} \\
\frac{I \subseteq J \quad T_i \leq T'_i \ (i \in I)}{\oplus \{\ell_i : T_i\}_{i \in I} \leq \oplus \{\ell_j : T'_j\}_{j \in J}}
\end{array}$$

Tolerance has three independent knobs: the existential $\diamond$ (§4.2), subtyping $\leq$ (here), and the granularity of α . “Flexible, tolerant of small details” is the profile $\langle \text{coarse } \alpha, \diamond, \text{linear } \mathcal{L} \rangle$; a later safety layer flips the same machinery to the universal $\square$ over permission predicates.

5 The Achievability Type System

Achievability factors into four premises, each separately decidable, combined by the top rule ACH.

5.1 Capability and goal typing

$$\begin{array}{c}
\text{T-ACT} \\
\frac{\Gamma(a) = \langle \text{pre}, \text{eff} \rangle \quad \varphi \models \text{pre}}{\Gamma \vdash \{\varphi\} a \{\text{sp}(\varphi, \text{eff})\}}
\end{array}
\qquad
\begin{array}{c}
\text{T-MISS} \\
\frac{a \notin \text{dom}(\Gamma)}{\Gamma \vdash a \triangleright \mathbf{X} \text{ (MISSING_CAPABILITY)}}
\end{array}$$

$\text{sp}(\varphi, \text{eff})$ is the strongest postcondition of the STRIPS effect. T-MISS fires on hallucinated planning: the plan names a tool the context does not grant, and achievability is refuted without any search.

5.2 The achievability judgment

$$\frac{\text{ACH} \quad \Gamma \supseteq \text{caps}(G) \quad G \Downarrow \{T_p\} \quad \forall p. \Gamma \vdash S_p \leq G \upharpoonright p \quad \Gamma; G \models \Diamond \varphi_{\text{goal}}}{\Gamma \vdash \{S_p\}_p : G \triangleright \Diamond \varphi_{\text{goal}}}$$

Read top-to-bottom: no hallucinated tools; the global type is realizable (no deadlocking handoff); every declared skill conforms to its projected contract up to subtyping; and the contract can reach the goal. The judgment is decided on G , and subtyping *transports* the verdict to the actual skills — legitimate by Theorem 2.

5.3 Reachability rules

The diamond is derived by the following inductive system, realized by the checker as a finite forward search.

$$\begin{array}{c} \text{REACH-GOAL} \\ \frac{W \models \varphi_{\text{goal}}}{\Gamma; (G, W) \models \Diamond \varphi_{\text{goal}}} \end{array} \quad \begin{array}{c} \text{REACH-STEP} \\ \frac{(G, W) \rightsquigarrow (G', W') \quad \Gamma; (G', W') \models \Diamond \varphi_{\text{goal}}}{\Gamma; (G, W) \models \Diamond \varphi_{\text{goal}}} \end{array}$$

$$\begin{array}{c} \text{REACH-OR} \\ \frac{\exists k \in I. \Gamma; (G_k, W) \models \Diamond \varphi_{\text{goal}}}{\Gamma; (p \triangleright \{\ell_i : G_i\}, W) \models \Diamond \varphi_{\text{goal}}} \end{array}$$

6 Metatheory

We separate what is *mechanized in Coq* (Theorems 1–3 and the concrete instance, all axiom-free) from what is *proved on paper* (decidability and the undecidability boundary).

6.1 Soundness of the abstraction

Let $\mathcal{A}$ be the abstract transition system the checker explores and $\text{abs} : W \rightarrow \mathcal{A}$ the abstraction. We require two simulation conditions:

$$(\text{step-sim}) \quad W \rightsquigarrow W' \Rightarrow \text{abs}(W) \hat{\rightsquigarrow} \text{abs}(W'), \quad (\text{goal-sim}) \quad \text{goal}(W) \Rightarrow \hat{\text{goal}}(\text{abs}(W)).$$

Lemma 1 (Transport). *If $(G, W_0) \rightsquigarrow^* (G', W)$ then $\text{abs}(W_0) \hat{\rightsquigarrow}^* \text{abs}(W)$.*

Proof. Induction on the closure; (STEP-SIM) discharges the step case. Mechanized as `reach_abs`. $\square$

6.2 Refutation soundness, tolerance, monotonicity

Theorem 1 (Refutation soundness). *If the abstract system has no reachable goal state from $\text{abs}(W_0)$, then no concrete run from W_0 reaches φ_{goal} . Hence a verdict of impossible is never wrong.*

Proof. Contrapositive of Lemma 1 composed with (GOAL-SIM). Mechanized, axiom-free: $\square$

```
Theorem refutation_sound : forall s0,
  (~ exists a, reach astep (abs s0) a /\ agoal a) ->
  (~ exists w, reach cstep s0 w /\ cgoal w).
(* Print Assumptions refutation_sound. => Closed under the global context *)
```

Theorem 2 (Tolerance soundness). *Let $\hat{\rightsquigarrow}_1 \subseteq \hat{\rightsquigarrow}_2$ be two abstract step relations over the same states (the coarser $\hat{\rightsquigarrow}_2$ has more edges), with the same goal predicate. If $\hat{\rightsquigarrow}_2$ cannot reach a goal state from $\text{abs}(W_0)$, neither can $\hat{\rightsquigarrow}_1$; composed with Theorem 1, a refutation computed on any coarser*

over-approximation transfers to the concrete system. Hence coarsening α to tolerate more detail never produces a false refutation. (Coq: *tolerance_sound*.)

Proof. Reachability is monotone in the step relation: if $R_1 \subseteq R_2$ then $\text{reach}(R_1) \subseteq \text{reach}(R_2)$, by induction on the reflexive–transitive closure — the reflexive case is immediate, and a step $u R_1 v$ is also a step $u R_2 v$ (mechanized as `reach_mono`). Contraposing: no goal state reachable in R_2 implies none reachable in R_1 . Taking R_1 to be the abstract image of the concrete system guaranteed by (STEP-SIM) and R_2 the coarser exploration relation, composing with Lemma 1 and (GOAL-SIM) yields the concrete claim exactly as in Theorem 1. $\square$

Theorem 3 (Capability monotonicity). *Let $\Gamma \subseteq \Gamma'$ agree on their common domain. If some run over Γ reaches φ_{goal} , then some run over Γ' reaches φ_{goal} . Granting tools never makes an achievable goal impossible. (Coq: *cap_monotone*.)*

Proof. Every transition of the Γ -system is a transition of the Γ' -system: R-MSG, R-SEL, R-REC do not consult Γ , and a WORLD-ACT step through $a \in \text{dom}(\Gamma)$ uses $\Gamma(a) = \Gamma'(a)$, so the same successor world is produced. Thus $\rightsquigarrow_{\Gamma} \subseteq \rightsquigarrow_{\Gamma'}$, and the claim is `reach_mono` applied to the witnessing run. At the judgment level the other premises are unaffected: projection and conformance do not mention Γ , and $\Gamma \supseteq \text{caps}(G)$ only becomes easier. Note the asymmetry: removing a capability can (and should) flip a verdict — that is the refutation direction the mutation tests of §9.2 exercise. $\square$

Concrete instance. The flight skill of §2, variant A, is mechanized as `FlightInstance`. We prove `flight_refuted` (no run reaches `booked ∧ confirmation_sent`, since no step touches `conf`) and `booking_reachable` (a booking is still reachable). Both are axiom-free, witnessing non-vacuity of Theorem 1.

6.3 Incompleteness, by design

Proposition 1 (Incompleteness). $\Gamma; G \models \Diamond \varphi_{\text{goal}}$ may hold while no concrete run reaches φ_{goal} : the witnessing abstract path can be spurious when α collapses a distinction that mattered. The system is sound for $\times$ and incomplete for $\checkmark$.

This is the price of decidable, tolerant over-approximation; tightening α trades tolerance for precision. The residue is confined to two edges, motivating the layered architecture:

- **Payload faithfulness** (bottom edge): a tool’s declared post-condition (“filters to fares < 500”) may be false — a runtime obligation for a deterministic monitor (Layer C), not a static one.
- **Intent fidelity** (top edge): the formalized goal may not capture what the user meant — irreducibly semantic, surfaced at the single compaction checkpoint for human review.

6.4 Decidability and the autonomy boundary

Theorem 4 (Decidability). *In the fragment with finitely many roles (static topology), tail-recursive G , decidable state logic $\mathcal{L}$, and $\mathcal{L}$ -definable effects, $\Gamma; G \models \Diamond \varphi_{\text{goal}}$ is decidable.*

Proof. We exhibit the decision procedure the implementation realizes and prove it terminating and sound.

Symbolic configurations. A symbolic configuration is a triple $\sigma = (c, P, \phi)$: a control point c in the protocol (a position in the finite step tree; by tail-recursion every μX body returns control to a fixed loop head, so the set C of control points is finite), a finite set $P \subseteq \text{Pred}$ of predicates currently

true (frame: all others false), and a path constraint $\phi \in \mathcal{L}$ over SSA-indexed numeric variables. σ over-approximates the set of worlds $\{\langle B, N \rangle : B = P, N \models \exists\text{-closure of } \phi\}$.

Search. Forward search from (c_0, P_0, init) : actions apply the STRIPS effect to P and conjoin assignment/nondeterminism constraints onto ϕ at fresh SSA indices; guards and goals are satisfiability queries $\phi \wedge \psi$ in $\mathcal{L}$, decidable by assumption; choices branch.

Widening at back edges. On traversing a back edge to loop head X , the procedure (i) *havocs* the numeric summary — replaces ϕ by $\top$ and advances every SSA index — and (ii) prunes if the pair (X, P) has been seen at a back edge before, else records it.

Termination. Between back edges the remaining step structure is a finite tree, so only finitely many queries occur. Each back-edge traversal either records a new pair (X, P) or prunes; there are at most $|C| \cdot 2^{|\text{Pred}|}$ such pairs, so the search terminates.

Soundness for refutation. Havoc widening only enlarges the denoted world set ($\top$ is entailed by any ϕ), so the widened system over-approximates the concrete one; by Theorem 2 a refutation on it transfers. Pruning is exact for the widened system: after widening, a configuration is fully determined by (X, P) — the constraint is always $\top$ and the continuation at a loop head is fixed by the (unique) binder — so a pruned re-visit explores exactly the state space already explored, and no goal-satisfying run is lost.

Definiteness. Within the fragment the procedure always exits with a witness or an exhausted search, never UNKNOWN; hence achievability is decidable. The price of the havoc is possible spurious witnesses *through* loops (a numeric refinement re-established each iteration is forgotten), which lands on the incomplete side of Proposition 1, never the sound side. $\square$

Theorem 5 (Undecidability under autonomy). *If skills may spawn participants at run time (unbounded $\mathcal{R}$) with dynamic channels, achievability is undecidable.*

Proof. By reduction from control-state reachability of communicating finite-state machines (CFSMs), undecidable already for two machines connected by unbounded FIFO channels [5]. Given a CFSM system (M_1, M_2) and a target control state q^* of M_1 , construct a skill system as follows. Each machine becomes a role whose local control states are encoded in branch structure; each FIFO channel is simulated by a stream of spawned single-message courier participants: to send letter m , a role spawns a fresh courier and hands it m ; couriers deliver in spawn order to the receiving role (dynamic channels give each courier its own session with the receiver, and an integer counter — expressible in $\mathcal{L}$ — sequences delivery, so FIFO order is preserved). Transitions of M_i on sending/receiving m become the corresponding spawn/receive steps; the world is used only for one marker predicate *hit*, added by a capability that M_1 's encoding invokes exactly upon entering q^* ; the goal is $\varphi_{\text{goal}} = \text{hit}$. The construction is computable, and runs of the skill system correspond bijectively to runs of the CFSM system, with *hit* reachable iff q^* is. A decision procedure for achievability in the unbounded-spawn fragment would therefore decide CFSM control-state reachability. Hence none exists, and a sound checker must degrade: refute what is refutable without bounding the topology (§6.5), and answer UNKNOWN otherwise. $\square$

The two theorems draw the line, and the line is autonomy itself. A session type's hard case is *non-local choice*; an autonomous agent's defining behaviour is to make such choices and spawn helpers on the fly. The feature that makes agents agentic is exactly the feature that crosses from decidable into undecidable.

6.5 The establisher closure: refutation that survives autonomy

Theorem 5 bounds *reachability analysis*, not every refutation. Some verdicts quantify over all protocols at once and therefore do not care whether the topology is static.

Lemma 2 (Establisher closure). *Let $\mathcal{E}(\Gamma, \text{init}) = \text{init} \cup \bigcup_{a \in \text{dom}(\Gamma)} \text{Add}_a$. For every protocol G over Γ — of any shape, including protocols that spawn unboundedly many participants, so long as every participant acts through Γ — and every reachable configuration (G', W) with $W = \langle B, N \rangle$: $\{p : B(p) = \text{tt}\} \subseteq \mathcal{E}(\Gamma, \text{init})$.*

Proof. Induction on $\rightsquigarrow^*$. Initially $B_0 = \text{init} \subseteq \mathcal{E}$. R-MSG, R-SEL, R-REC (and participant spawning) leave W untouched. For R-ACT through a , $B' = (B \cup \text{Add}_a) \setminus \text{Del}_a \subseteq B \cup \text{Add}_a \subseteq \mathcal{E} \cup \text{Add}_a = \mathcal{E}$. $\square$

Corollary 1 (Protocol-independent refutation). *Let $\widehat{\varphi_{\text{goal}}}$ be φ_{goal} with every predicate outside $\mathcal{E}(\Gamma, \text{init})$ replaced by $\perp$, every remaining predicate by a fresh free boolean, and every arithmetic comparison by a fresh free boolean. If $\widehat{\varphi_{\text{goal}}}$ is unsatisfiable, then no run of any protocol over Γ satisfies φ_{goal} — in particular the verdict is sound outside the decidable fragment of Theorem 4.*

Proof. Suppose some reachable W satisfied φ_{goal} . By Lemma 2 every predicate outside $\mathcal{E}$ is false in W , matching the $\perp$ substitution; valuing each remaining free boolean by its truth in W yields a model of $\widehat{\varphi_{\text{goal}}}$, contradicting unsatisfiability. $\square$

The checker decides Corollary 1 with one SMT query before anything topology-dependent, so a skill that spawns helpers *and* pursues a goal conjunct nothing in Γ establishes is still refuted — with the dead conjuncts as the certificate — rather than surrendered to UNKNOWN. This generalizes the **FlightInstance** argument of the mechanization from one spec to a decision rule. It also sharpens the vocabulary of verdicts: GOAL_UNSAT via the closure refutes *every* plan over these tools, while GOAL_UNSAT from the search refutes *this* plan; the two carry different repair advice (acquire a tool vs. fix the plan).

6.6 The tri-partition

Corollary 2. *The compiler reduces “will the goal be achieved” to three obligations on disjoint substrates:*

Concern	Where	Mechanism
goal achievability	static, decidable	Coq-proved may-reachability; ms; no LLM
per-step safety / least privilege	Layer B (future)	same engine, $\square$ + permission tiers
payload faithfulness	Layer C, runtime	deterministic monitors with blame
intent fidelity	top edge, synthesis	one human checkpoint

7 Strengthening the Discipline

Compiling real, deployed skills (§9.2) exposed three places where the discipline as stated either refuted skills that are perfectly executable or answered less than it could. Each fix is a small extension with its own soundness argument; none touches the trusted core’s guarantee.

7.1 Observed choice: coordination inside a conversation

Compacting real consumer skills, every initial NON_PROJECTABLE refutation traced to the same modelling mismatch: a choice resolved *inside a live conversation* (the user picks an option in chat; the business answers on a phone call), where the classic unobserved-choice condition does not apply because the medium announces the outcome to every participant. PROJ-OBS (§4.3) captures this.

Proposition 2 (Soundness of observed choice). *Let $D(G)$ desugar every $\mathbf{p} \triangleright^{\text{obs}} \{\ell_i : G_i\}$ into $\mathbf{p} \triangleright \{\ell_i : \mathbf{p} \rightarrow \mathbf{q}_1 : \ell_i. \dots \mathbf{p} \rightarrow \mathbf{q}_k : \ell_i. G_i\}$, prefixing each branch with an explicit broadcast of the selected label to the other roles $\mathbf{q}_1, \dots, \mathbf{q}_k$. Then (i) $D(G)$ is projectable by PROJ-BRN exactly where G is projectable by PROJ-OBS, with identical projections up to the added receive prefixes; and (ii) D preserves may-reachability, since R-MSG steps do not change the world. Consequently checking G with PROJ-OBS equals checking a standard global type whose communication the conversation medium performs implicitly.*

Proof. (i) Under D , every branch of the choice begins, for each non-selector $\mathbf{q}$, with the distinguishing receive $\mathbf{p} ? \ell_i$, which is precisely the premise of PROJ-BRN; the resulting local type is $\&\{\ell_i : T_i\}$ with T_i the projection of the branch body — the PROJ-OBS result. For the selector both give $\oplus\{\ell_i : \cdot\}$ over the same continuations. (ii) is immediate from R-MSG: inserted messages neither guard nor affect W , so runs and goal-satisfaction correspond one-to-one. $\square$

The observation behind the fix is worth stating on its own: *unobserved choice is a ≥ 3 -party asynchronous phenomenon*. In a binary synchronous session the selection is the communication; only when a third party must act on a branch it never hears about does realizability fail. Compaction should therefore mark conversation-embedded choices observed, and reserve explicit **msg** steps for genuinely asynchronous handoffs between separate agents.

7.2 Adversarial achievability: when the environment picks

The diamond of §4.2 is *angelic*: every choice, including one resolved by an external party, is existential. A skill whose goal is reachable only if the business says yes is angelically achievable — true, but weaker than an operator often wants to know. We add a *must* mode. Choices may be annotated *external*; define $\models^{\text{adv}}$ by replacing REACH-OR on external choices with its universal dual:

$$\frac{\text{REACH-AND-EXT} \quad \forall k \in I. \Gamma; (G_k, W) \models^{\text{adv}} \Diamond \varphi_{\text{goal}}}{\Gamma; (\mathbf{p} \triangleright^{\text{ext}} \{\ell_i : G_i\}, W) \models^{\text{adv}} \Diamond \varphi_{\text{goal}}}$$

so the agent’s own choices remain existential while the environment’s are demonic: AND-OR reachability, i.e. the existence of a winning strategy for the agent coalition against the declared model.

Proposition 3 (Compositional refutation soundness). *An adversarial refutation is sound relative to the declared model: if the checker reports $\Gamma; G \not\models^{\text{adv}} \Diamond \varphi_{\text{goal}}$, then for every agent strategy there is a resolution of external choices under which the goal is not reached.*

Proof. By induction on the search tree. At an external choice the checker refutes by exhibiting a branch ℓ_k whose subtree it refutes; the environment plays ℓ_k . At the leaves and along internal structure, refutation of a subtree is may-refutation, which is sound by Theorem 1: no run of that subtree reaches the goal at all, in particular none under the chosen strategy. The induction glues the environment’s branch selections into a single spoiling strategy. $\square$

The asymmetry is preserved but flipped in an instructive way: adversarial *achievable* inherits the incompleteness of Proposition 1 per branch, while adversarial *refutation* composes may-refutations and stays trustworthy. The same annotation vocabulary is the natural substrate for the Layer-B $\square$ analysis anticipated in §4.4.

7.3 Counterexample-guided compaction repair

The trust boundary makes compaction errors *inspectable*; it does not by itself make them *cheap to fix*. In practice the dominant compaction artifact is structural (a conversation modelled as an unannounced handoff), and the checker’s refutation names it precisely. We therefore close a bounded loop at the boundary: when the trusted checker refutes with `NON_PROJECTABLE`, the counterexample is returned to the untrusted compactor with a repair instruction restricted to communication structure — it may mark a choice observed or add the informing messages the prose actually describes, and may *not* add capabilities, alter effects, or weaken the goal. The candidate re-enters through the same schema gate and checker. Soundness is untouched — every verdict is still computed by the trusted core on the pack in front of it — and the restriction matters: repairing `MISSING_CAPABILITY` or `GOAL_UNSAT` this way would invite the model to invent tools or dilute the goal, exactly what rule 1 of the compaction contract forbids. On the real-skill corpus one round of repair resolves every structural artifact (§9.2).

7.4 Admission control at the boundary

Orthogonally to the type discipline, the compiler’s inputs are *bundles* — prose, scripts, manifests, permission metadata — and honesty of the declared Γ begins with the bundle not being hostile. A deterministic SkillSpector-style pre-pass [10] vets manifest consistency, invisible/bidirectional Unicode, instruction-injection patterns in descriptions and hidden comments, pipe-to-shell and decode-and-execute patterns in embedded code, and consistency of prose tool use with declared `allowed-tools`. It is admission control, not typing: it asks whether the bundle should be trusted enough to compile at all.

8 Implementation

The compiler is a Python package (`skillc`) deciding all four premises of ACH: capability soundness, realizability by actual projection with merge (`PROJ-SEL/BRN/MRG/OBS`), conformance by coinductive Gay–Hole subtyping (`SUB-EXT/INT`), and may-reachability over Z3 with the procedure of Theorem 4 — tail-recursive loops explored with predicate-state saturation and havoc widening at back edges, dynamic spawning degrading to `UNKNOWN` after the closure certificate of Corollary 1 has been tried. Front-ends produce packs three ways: a deterministic markdown compactor (frontmatter and prose tool extraction with line-number provenance, judged against pluggable capability profiles of the target runtime), an embedded-pack escape hatch, and the untrusted LLM compactor with the schema gate and the bounded repair loop of §7.3. The Coq development (Theorems 1–3 plus the concrete instance) checks under Coq 8.18 with no axioms, verified by `Print Assumptions`; it certifies the trusted core’s soundness argument, while the compiler itself is the Python implementation. On success the checker returns the witness path; on refutation, the failing premise and its frontier (missing capabilities, unsatisfiable guard, dead goal conjuncts, the non-projectable role, or the non-conforming skill).

9 Evaluation

9.1 Ground-truth corpus

We assembled a corpus of 15 skills spanning the canonical failure modes — hallucinated planning, retry-forever loops, coordination deadlock, refinement violations — with achievable controls (including detours and informed branches) and two deliberately *spurious* cases exercising the deferred edges. Each skill carries a ground-truth semantic label; the positive class is *achievable*.

	predicted achievable	predicted impossible
truly achievable	TP = 6	FN = 0
truly impossible	FP = 2	TN = 7

Two claims, both confirmed. **Soundness:** FN = 0 — no achievable goal was wrongly refuted, the empirical face of Theorem 1 (achievability recall $6/6 = 100\%$). **Incompleteness, contained:** FP = 2, and both are exactly the planted spurious cases — one a false payload post-condition (Layer-C obligation), one an under-specified goal (intent edge). The only way to fool the checker is through the two residues it openly defers; no structural failure was missed. Each refutation reason fires on its matching mode:

Failure mode (source)	Verdict reason
hallucinated planning — invokes a non-existent tool	MISSING_CAPABILITY
hallucinated planning — no establisher for a goal conjunct	GOAL_UNSAT
retry-forever loop — guard never satisfiable	BLOCKED_GUARD
coordination deadlock — unobserved choice / bad handoff	NON_PROJECTABLE
refinement violation — budget unsatisfiable on every run	GOAL_UNSAT

A second, six-spec corpus exercises what the extensions add: tail-recursive retry with an exit (achievable, witness through the back edge), a loop that never establishes its goal (refuted by saturation), spawning with and without a dead goal conjunct (UNKNOWN vs. the closure refutation of Corollary 1), and conformance in both directions (a handler declaring an *extra* external branch passes by SUB-EXT; one dropping a contracted branch is refuted NON_CONFORMANT).

9.2 Real, deployed skills

The proof-of-concept corpus answers “does the checker match the theory”; the sharper question is whether the *compiler* validates the goal achievability of skills people actually run. We evaluate on Anthropic’s public skills corpus (32 SKILL.md bundles: consumer workflows such as booking-by-phone and grocery ordering, and document-processing skills), in three regimes.

Capability level, deterministic. The deterministic front-end compacts each skill against a pluggable *capability profile* of the target runtime. Under the consumer-runtime profile, **32/32** skills are achievable — zero false alarms on deployed skills, the empirical face of Theorem 1. Under a coding-CLI profile, **16/32** are refuted MISSING_CAPABILITY with the exact absent tool and source line (e.g. `ask_user_input_v0`, a consumer-app widget no CLI grants): the same skill, two capability contexts, two verdicts — achievability is a judgment about $\langle \text{skill}, \Gamma \rangle$, never the skill alone. Mutation testing closes the loop in both directions: removing one invoked tool from the profile flips every affected skill to MISSING_CAPABILITY naming exactly that tool, and granting the named frontier back flips it to achievable (Theorem 3 on real data).

Semantic level, live compaction. Four representative consumer skills were compacted by a live LLM into semantic packs (goals such as *booked* $\wedge$ *calendar-updated* $\wedge$ *user-informed* with per-step guards), through the schema gate and, where needed, one round of the §7.3 repair loop. All four check ACHIEVABLE — no false alarm on deployed skills at the semantic level either.

Seeded semantic faults. Against each achievable semantic pack we seed the two canonical wounds with known ground truth: *drop* a capability the plan invokes, and *strip* a goal conjunct’s

establishers from every tool that provides it. The compiler refuted **6/6** mutants, naming the dropped tool (`MISSING_CAPABILITY`, `frontier`) or the dead conjunct (`GOAL_UNSAT` via Corollary 1) in every case. The admission pre-pass, run over the same 32 real bundles, reports zero error-severity findings, while a planted poisoned bundle (invisible Unicode, description injection, hidden HTML-comment instruction, pipe-to-shell install) trips every detector class.

10 Related Work

Prior-art note. The claims below reflect the authors’ survey at submission time; the fusion’s novelty should be confirmed against a systematic search of the planning-meets-behavioural-types literature, which is dispersed across communities.

Multiparty session types [1, 2] supply projection, deadlock-freedom, and subtyping [3]. We reuse this for the realizability premise but add a world and a goal, which classical session types do not model. *Automated planning* [4] supplies capability-guarded goal reachability; we reuse its frame semantics but embed it in a multiparty choreography. The contribution is the *fusion*, decided over an LLM-synthesized, goal-marked global type. Recent agent-verification work is adjacent but differently aimed: *provable coordination via message sequence charts* [7] projects a global choreography for LLM agents but targets coordination correctness, not goal achievability; *VeriGuard* [8] separates offline policy verification from online monitoring as a safety shield; *agent behavioural contracts* [9] bring design-by-contract to single agents.

Skill-bundle scanners and verified-skill catalogs are complementary. SkillSpector [10] and the NVIDIA verified-agent-skills pipeline [11] address the same deployment object — portable agent skills — but ask a different question: a pre-publication security control normalizes a bundle and runs deterministic scanners (static pattern detectors, AST and taint analyzers, manifest-consistency and metadata-poisoning checks, supply-chain checks). Valuable admission control, but not a compiler or type discipline: it defines no operational semantics of goal-marked skills, projects nothing, decides no reachability, proves no refutation soundness. In our architecture such a pass sits at the compiler’s boundary (§7.4): before compaction it vets the incoming bundle; after compaction it supports the honesty of declared capabilities. The type checker then consumes the sanitized pack and proves the narrower claim that the declared goal is not structurally impossible. None of the above frames the static check as goal achievability over a synthesized protocol, nor gives the refutation-sound / achievement-incomplete asymmetry with a mechanized proof. Finally, the trust boundary connects to learning with verifiable rewards: the checker’s sound *impossible* verdicts are cheap, on-distribution negative labels — a reward signal needing no LLM judge — a loop we leave to future work.

11 Limitations and Future Work

- **Relative soundness.** Everything is proved about the declared Γ and S . If the prose lies (a “read-only” tool that writes), the checker verifies a fiction; honest-declaration is a separate obligation, of which this makes only the interaction slice decidable.
- **Static topology for totality.** Dynamic spawning drops the procedure to a semi-decision (Theorem 5); a practical system answers UNKNOWN rather than guessing.
- **Trusted abstraction.** A coarse α silently widens tolerance (more false achievable), never breaking Theorem 1 but weakening usefulness.
- **Engineering gaps.** The mechanization covers soundness of abstract reachability (Theorems 1–3); the newer results — Lemma 2, Theorem 4’s procedure, Propositions 2 and 3 — are proved on paper here and their mechanization is routine future work. The merge $\sqcap$ implements label-union on external branches with structural recursion elsewhere — a sound core of, not

the complete, merge lattice; havoc widening forgets *all* numeric state at back edges (always in the tolerant direction). The real-skill corpus is one vendor’s public catalog (32 bundles) and the semantic-level evaluation covers four of them with one compaction model; scaling both is next.

- **Compaction quality is a first-class variable.** The real-skill study shows the checker’s verdict is only as meaningful as the pack: the two systematic artifacts we found (under-granted runtime abilities, unannotated conversational choices) both produce false refutations — safe-side errors, surfaced with counterexamples, and fixable by prompt contract and bounded repair — but a compaction study across models and prompt contracts is needed before quantitative claims about the pass direction generalize.

12 Conclusion

An LLM can draft an agent’s plan from intent; the open question has been whether anything can tell, cheaply and before execution, that the plan is doomed. For the *achievability* question the answer is yes, with a guarantee: fuse planning’s capability-guarded reachability with session types’ projectability over a goal-marked global type, treat the LLM’s compaction as untrusted, and prove the checker sound for refutation. The verdict *impossible* is then never wrong; *achievable* is honest about the two edges it defers. The same machinery, with the diamond flipped to a box, extends upward to per-step safety — a layered route from “can this agent even do that?” to “will it do it safely?”

References

- [1] K. Honda, N. Yoshida, M. Carbone. Multiparty Asynchronous Session Types. *J. ACM* 63(1), 2016.
- [2] E. Li, F. Stutz, T. Wies, D. Zufferey. Complete Multiparty Session Type Projection with Automata. *CAV* 2023.
- [3] S. Gay, M. Hole. Subtyping for Session Types in the Pi Calculus. *Acta Informatica* 42(2–3), 2005.
- [4] R. Fikes, N. Nilsson. STRIPS: A New Approach to the Application of Theorem Proving to Problem Solving. *Artif. Intell.* 2(3–4), 1971.
- [5] D. Brand, P. Zafriopulo. On Communicating Finite-State Machines. *J. ACM* 30(2), 1983.
- [6] H. G. Rice. Classes of Recursively Enumerable Sets and Their Decision Problems. *Trans. AMS* 74, 1953.
- [7] Provable Coordination for LLM Agents via Message Sequence Charts. arXiv:2604.17612, 2026. [verify]
- [8] VeriGuard: Enhancing LLM Agent Safety via Verified Code Generation. arXiv:2510.05156, 2025. [verify]
- [9] Agent Behavioral Contracts: Formal Specification and Runtime Enforcement. arXiv:2602.22302, 2026. [verify]
- [10] N. Paz et al. SkillSpector: A Pre-Publication Security Control for Agent Skills. OpenReview, 2026. [verify]
- [11] NVIDIA. NVIDIA-Verified Agent Skills Provide Capability Governance for AI Agents. NVIDIA Technical Blog, 2026. [verify]